ChatGPT

FailureCloud Implementation Plan

FailureCloud should be built as a **scenario compiler and evaluation layer** for robotics, not as a new standalone simulator. The core product is a structured `scenario.json` that can be generated from natural language, validated, previewed, executed first in PyBullet, and then exported into downstream robotics and perception formats such as ROS 2 messages, OpenPCDet, MMDetection3D, and Isaac-compatible assets. PyBullet is the right MVP backend because it already supports loading URDF/SDF/MJCF assets, ray queries, and camera rendering, while future adapters can target higher-fidelity backends such as Isaac Sim, CARLA, Gazebo, Nebius Workbench, and Cosmos-enhanced pipelines. [1]

Product goals and MVP

The goal of FailureCloud is to convert a user's natural-language description of a rare or safety-critical robotics edge case into an **executable robot test case** with: world/task specification, preview, sensor outputs, auto-labels, reward/success metrics, and exports. This is useful because simulation platforms and synthetic-data tools already exist, but teams still spend substantial time hand-authoring tasks, reward functions, validation logic, and export plumbing. Isaac Sim and Omniverse Replicator explicitly position synthetic-data generation as valuable for robotics and autonomous systems, especially for difficult long-tail cases and perfect labels; FailureCloud should sit one layer above that capability. [2]

Recommended MVP scope

In scope for MVP	Out of scope for MVP
Prompt → `scenario.json` using Claude structured outputs	High-fidelity sensor physics
JSON-schema validation and deterministic normalization	Real fluid simulation
Browser preview with Three.js	Full Isaac/CARLA/Gazebo runtime adapters
PyBullet execution backend	End-to-end managed training on Nebius
RGB, depth, LiDAR-like raycast, segmentation/instance IDs, collisions	Production-grade sim-to-real transfer claims
Auto-labels, reward computation, success/failure report	Large-scale distributed generation
Exporters: PyBullet bundle, ROS-style folder, OpenPCDet, MMDetection3D-lite, Nebius/Cosmos manifests	Full rosbag2 writer if time is short

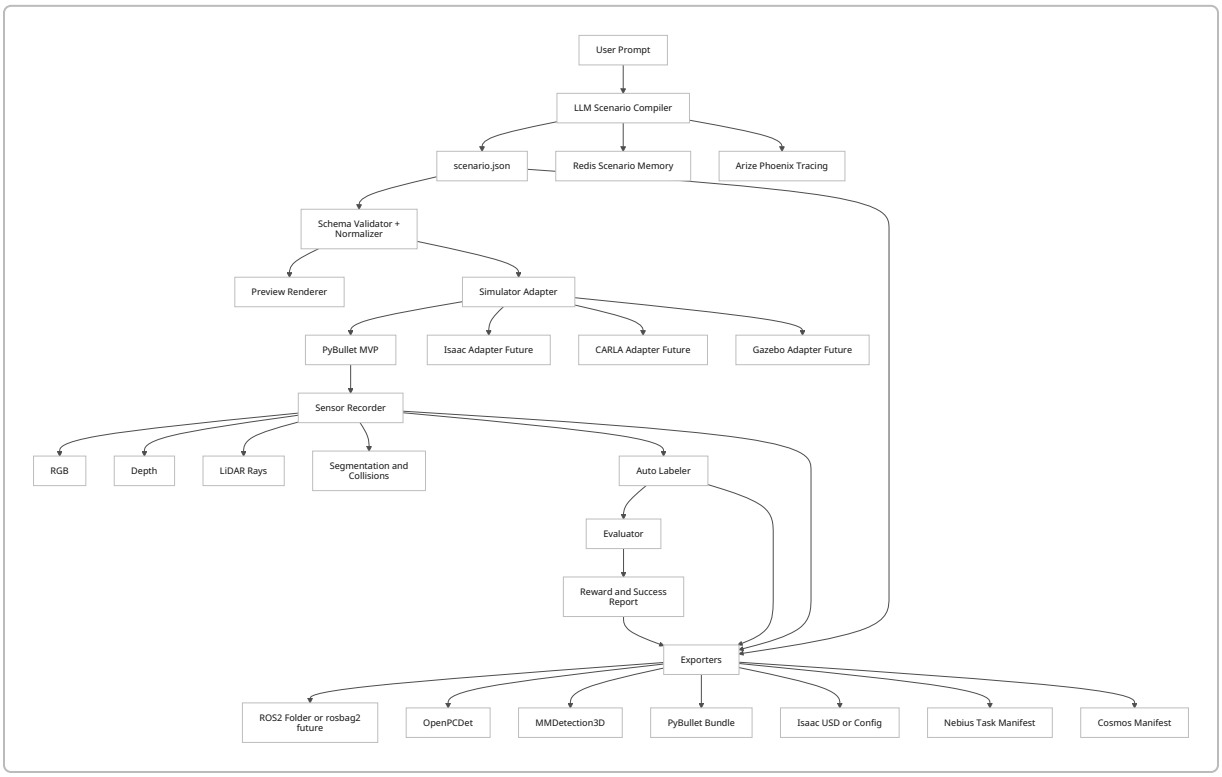
Success criteria for MVP

A successful MVP should do the following in one end-to-end demo:

1. Accept a prompt like “carry a cup of water across a slippery floor while avoiding a dropped box.”
2. Produce a valid `scenario.json`.
3. Render a browser preview and run the same scenario in PyBullet.
4. Record RGB, depth, LiDAR-like points, object poses, and collisions.
5. Compute `water_left_percent`, reward, and pass/fail.
6. Export at least one real downstream format that another tool can consume.

Architecture and data contracts

PyBullet provides the MVP execution substrate because Bullet’s official quickstart describes PyBullet as supporting URDF, SDF, and MJCF loading, forward/inverse dynamics, collision detection, and ray intersection queries; Bullet’s examples also show `rayTestBatch` and `getCameraImage` for synthetic LiDAR-style scans and RGB/depth/segmentation output. ROS 2 `PointCloud2`, `Image`, and `CameraInfo` define the canonical robotics-facing export contract. OpenPCDet expects `.npy` point clouds and `.txt` labels in the format `[x y z dx dy dz heading_angle category_name]`, while MMDetection3D’s custom-dataset guidance recommends converting point clouds to `.bin` and organizing annotations in a KITTI-like or basic custom format. Isaac Sim can publish RTX LiDAR as `sensor_msgs/PointCloud2` or `LaserScan`, with optional metadata such as intensity and object IDs, and CARLA exposes ray-cast LiDAR, semantic LiDAR, segmentation, and instance outputs. [3]



Core file and schema definitions

Recommended canonical bundle:

```
failurecloud_output/
  scenario.json
  validation_report.json
  reward_config.json
  preview/
    scene_snapshot.json
  sensor_data/
    rgb/000000.png
    depth/000000.npy
    seg/000000.npy
    lidar/000000.npy
  labels/
    000000.json
  eval/
    episode_000.json
    success_metrics.json
  exports/
    pybullet/
    ros/
    openpcdet/
    mmdet3d/
    isaac/
    nebius/
    cosmos/
```

Recommended `scenario.json` contract:

```
{
  "schema_version": "0.1.0",
  "scenario_id": "fc_20260620_001",
  "name": "slippery_cup_carry",
  "domain": "warehouse_robotics",
  "environment": {
    "type": "warehouse",
    "lighting": "dim",
    "weather": "none",
    "physics": {
      "gravity": [0, 0, -9.81],
      "floor_friction": 0.25,
      "time_step": 0.0041667
    }
```

```
  },
  "robot": {
    "type": "mobile_base",
    "asset_ref": "robots/mobile_base.urdf",
    "start_pose": {"position": [0, 0, 0], "yaw": 0.0},
    "goal_pose": {"position": [5, 0, 0], "yaw": 0.0}
  },
  "objects": [
    {
      "id": "cup_1",
      "class": "cup",
      "asset_ref": "objects/cup.urdf",
      "pose": {"position": [0.2, 0.0, 0.8], "yaw": 0.0},
      "properties": {"contains": "water", "initial_water_percent": 100}
    },
    {
      "id": "box_1",
      "class": "obstacle",
      "asset_ref": "objects/box.urdf",
      "pose": {"position": [2.5, 0.3, 0.2], "yaw": 0.0}
    }
  ],
  "dynamic_actors": [
    {
      "id": "human_1",
      "class": "pedestrian",
      "trajectory": [[3.0, -1.0, 0.0], [3.0, 1.0, 0.0]]
    }
  ],
  "sensors": {
    "rgb_camera": {"enabled": true, "width": 640, "height": 480, "fov_deg": 75},
    "depth_camera": {"enabled": true, "width": 640, "height": 480, "near":
0.02, "far": 20.0},
    "lidar": {"enabled": true, "num_rays": 1024, "range_m": 12.0, "height_m":
0.5}
  },
  "task": {
    "type": "carry_object_to_goal",
    "termination": {"timeout_s": 30, "on_collision": false},
    "success": {
      "goal_reached": true,
      "min_water_left_percent": 70,
      "max_collisions": 0
    }
  },
  "exports": ["pybullet", "ros2_folder", "openpcdet", "mmdet3d_basic"]
}
```

Validation rules

Recommended hard validation rules:

Rule	Why
`schema_version` required	Migration safety
Every object/actor ID globally unique	Labeling/export consistency
Sensor configs must include dimensions and ranges	Exporters need full calibration
Robot and goal poses must be present	Evaluator requires target state
Reward and success fields required	No scenario without measurable outcome
Physics values bounded	Prevent invalid sims
Asset refs must resolve or map to placeholder assets	Preview/sim determinism
Dynamic actor trajectories must be time-normalized	Replay determinism
Export names must be supported by installed adapters	Build-time correctness

Recommended soft validation rules:

Rule	Action
Missing object dimensions	Infer from asset metadata
Missing LiDAR intensity model	Default to `distance_decay`
Missing class map	Attach default ontology
Missing camera intrinsics	Derive from width, height, FOV

Adapter and export comparison

Adapter	MVP or future	Native strengths	Scenario mapping difficulty	Output fidelity
PyBullet	MVP	Fast setup, ray tests, RGB/depth/segmentation, RL loops	Low	Medium-low
Isaac Sim / Isaac Lab	Future	RTX LiDAR, ROS2 bridge, USD, Replicator, RL framework	Medium	High
CARLA	Future	AV maps, traffic, LiDAR, semantic LiDAR, cameras	Medium	High for AV
Gazebo	Future	ROS-native robotics workflows, SDF sensors, GPU lidar	Medium	Medium

Adapter	MVP or future	Native strengths	Scenario mapping difficulty	Output fidelity
Nebius Workbench	Future	Workflow orchestration, manifests, artifacts, cloud pipeline	Medium-high	Depends on backend
Cosmos backend	Future	Photoreal world transformation / rare edge-case generation	High	High visual fidelity

Export target	Canonical files	Best downstream use
PyBullet bundle	`scenario.json`, `run_sim.py`, local assets	Demo replay, debugging
ROS 2 folder	points/images/calibration/transforms	Robotics stack replay
OpenPCDet	`.npy` points, `.txt` labels, `ImageSets`	LiDAR 3D detection training
MMDetection3D basic	`.bin` points + annotations/config	3D detection experimentation
Isaac	`.usd` or config manifest	High-fidelity re-execution
Nebius	YAML/JSON manifest + artifact refs	Pipeline orchestration
Cosmos	scene/video manifest + control inputs	High-fidelity world generation

Sequenced build plan

The implementation should be handed to engineers as a sequence of ten deliverable steps. Estimated hours below assume one strong full-stack/ML engineer; parallel teams can compress the calendar significantly.

Step	Inputs	Outputs	Acceptance criteria	Effort	Key implementation notes
Foundation repo setup	Product brief	Monorepo, environment files, stub services	`make dev` boots frontend + backend + worker	8h	Recommended stack: FastAPI backend, worker process, React/ Next.js frontend, shared `schemas/` package

Step	Inputs	Outputs	Acceptance criteria	Effort	Key implementation notes
Canonical schemas	Requirements above	JSON Schemas for scenario, labels, eval, exports	Schemas validate sample fixtures; CI passes	12h	Use versioned JSON Schema files plus Pydantic models
Claude prompt-to-scenario compiler	Prompt + schema	`scenario.json` generator endpoint	20 representative prompts produce valid JSON with <5% hard-validation failures	16h	Use Claude Structured Outputs or strict tool use so outputs are schema-conformant. Anthropic explicitly supports JSON Schema-validated structured outputs and strict tool inputs. [4]
Validator + normalizer	Raw scenario	Normalized scenario, validation report	Invalid scenarios produce actionable errors; soft rules auto-fill defaults	10h	Separate “reject” and “autofix” rules
Browser preview	Valid scenario	Three.js preview scene	User can see robot, obstacles, goals, and sensor frustum without running sim	18h	Start with primitives and placeholder assets; do not block on URDF rendering
PyBullet adapter	Normalized scenario	Spawned scene + simulation loop	Robot, floor, obstacles, and sensors load consistently	24h	Map materials to friction and assets to URDF or primitives

Step	Inputs	Outputs	Acceptance criteria	Effort	Key implementation notes
Sensor recorder	Simulation state	RGB, depth, segmentation, LiDAR arrays	One run produces synchronized frames and timestamps	20h	Use `getCameraImage` and `rayTestBatch`; Bullet examples show both patterns directly. [5]
Auto-labeler + evaluator	Sim frames + world state	Per-frame labels and episode metrics	3D boxes, classes, trajectories, collisions, reward, success report present and internally consistent	18h	Labels come from simulator state; point labels can attach hit object IDs from LiDAR rays
Exporters	Recorded bundle	ROS/OpenPCDet/ MMDet/PyBullet/ Nebius/Cosmos outputs	Export validation suite passes; sample downstream tools recognize files	22h	ROS exporter can be folder-first for MVP; rosbag2 optional
Observability, CI, demo polish	A working pipeline	Trace dashboards, scenario memory, smoke tests, deploy artifacts	End-to-end demo works from clean deploy; traces visible in Arize; scenario retrieval works in Redis	18h	Instrument prompt compile, validation, run, export spans with Phoenix OTLP; use Redis vector search for “similar scenario” retrieval. [6]

Key implementation snippets

Recommended Claude request pattern for schema-safe scenario generation:

```
from anthropic import Anthropic

client = Anthropic()

response = client.messages.create(
```

```
    model="claude-sonnet-4-6",
    max_tokens=2048,
    cache_control={"type": "ephemeral"},
    tools=[{
        "name": "emit_scenario",
        "description": "Return a robotics scenario JSON matching the required
schema.",
        "input_schema": SCENARIO_JSON_SCHEMA,
        "strict": True
    }],
    messages=[{
        "role": "user",
        "content": "Generate a warehouse robot task carrying a cup across a
slippery floor."
    }]
)
```

Anthropic's docs support tool schemas with JSON Schema, strict tool use for guaranteed schema-conforming tool inputs, and prompt caching for repeated prefixes. [7]

Recommended PyBullet sensor loop:

```
import math
import numpy as np
import pybullet as p

def lidar_scan(origin_xyz, num_rays=1024, ray_len=12.0, z=0.5):
    ray_from, ray_to = [], []
    for i in range(num_rays):
        a = 2.0 * math.pi * float(i) / num_rays
        ray_from.append([origin_xyz[0], origin_xyz[1], z])
        ray_to.append([
            origin_xyz[0] + ray_len * math.sin(a),
            origin_xyz[1] + ray_len * math.cos(a),
            z
        ])
    results = p.rayTestBatch(ray_from, ray_to)
    pts = []
    for hit in results:
        hit_uid = hit[0]
        hit_pos = hit[3]
        if hit_uid >= 0:
            x, y, z = hit_pos
            intensity = 1.0
            pts.append([x, y, z, intensity, hit_uid])
    return np.asarray(pts, dtype=np.float32)
```

```
def camera_frame(width=640, height=480, near=0.02, far=20.0):
    view = p.computeViewMatrix([0, 0, 1.0], [1, 0, 1.0], [0, 0, 1])
    proj = p.computeProjectionMatrixFOV(75, width / height, near, far)
    images = p.getCameraImage(width, height, view, proj,
renderer=p.ER_TINY_RENDERER)
    rgb = np.reshape(images[2], (height, width, 4))[..., :3]
    depth_buf = np.reshape(images[3], (width, height))
    depth = far * near / (far - (far - near) * depth_buf)
    seg = np.reshape(images[4], (width, height))
    return rgb, depth, seg
```

Bullet's official examples use `rayTestBatch` and `getCameraImage`, including depth linearization from the depth buffer. [5]

Recommended fake water model and reward:

```
def update_water(cup_tilt_deg, cup_accel, collision, dt, state):
    spill = 0.0
    if cup_tilt_deg > 15:
        spill += (cup_tilt_deg - 15.0) * 0.04 * dt
    if cup_accel > 2.0:
        spill += (cup_accel - 2.0) * 0.03 * dt
    if collision:
        spill += 8.0
    state["water_left_percent"] = max(0.0, state["water_left_percent"] - spill)
    return state

def compute_reward(progress, collision, water_left_percent, done, goal_reached):
    r = 1.0 * progress
    if collision:
        r -= 10.0
    r -= 0.02 * (100.0 - water_left_percent)
    if done and goal_reached and water_left_percent >= 70.0:
        r += 20.0
    return r
```

Recommended OpenPCDet label row:

```
x y z dx dy dz heading_angle category_name
2.50 0.30 0.20 0.50 0.50 0.40 0.00 Obstacle
0.20 0.00 0.80 0.10 0.10 0.20 0.00 Cup
```

This matches OpenPCDet's documented custom label format. [8]

Adapters, integrations, validation, and exports

Prompt-to-scenario logic

Use **Claude as the scenario compiler**, not as the simulator. The recommended flow is:

1. System prompt defines ontology, allowed object classes, sensor options, and physics bounds.
2. Claude emits strict schema-conforming JSON.
3. Validator normalizes missing defaults and returns explicit repair errors if needed.
4. Optional second LLM pass can generate human-readable explanation, failure hypotheses, and "harder variant" suggestions.

Anthropic's Structured Outputs and strict tool use are the right primitives because they provide guaranteed JSON-shape compliance; prompt caching is useful because the bulk of the task prompt prefix stays constant across many scenario requests. [9]

Sensor, labeling, and export rules

Recommended per-frame contracts:

- `sensor_data/lidar/*.npy`: `N x 5` float32 arrays as `[x, y, z, intensity, object_id]`
- `sensor_data/rgb/*.png`: RGB images
- `sensor_data/depth/*.npy`: float32 metric depth
- `sensor_data/seg/*.npy`: instance or body IDs
- `labels/*.json`: object-centric frame labels
- `eval/*.json`: reward, done, success metrics, failure reasons

PyBullet's camera path naturally exposes RGB, depth, and segmentation buffers; Bullet's notebook/quickstart materials describe the segmentation mask as containing body IDs for visible objects, which is sufficient for an MVP auto-labeling pass. [10]

Recommended ROS 2 export mapping:

FailureCloud field	ROS 2 output
LiDAR points	`sensor_msgs/PointCloud2`
Point fields	`x`, `y`, `z`, `intensity` `PointField`s
RGB image	`sensor_msgs/Image`
Depth image	`sensor_msgs/Image`
Camera intrinsics	`sensor_msgs/CameraInfo`
Robot and sensor transforms	`/tf` style transform export

`PointCloud2` is defined as a structured binary blob with point layout described by `fields`; common field names include `x`, `y`, `z`, `intensity`. `Image` contains `height`, `width`, `encoding`, `step`, and `data`, while `CameraInfo` carries `K`, `R`, `P`, and distortion metadata. [11]

Future simulator adapters

Isaac Sim and Isaac Lab should be the first future adapter because NVIDIA explicitly supports RTX LiDAR publishing to ROS 2 as `PointCloud2` / `LaserScan`, object IDs in point-cloud metadata, Replicator-based synthetic-data writers, and USD-based scene composition. Isaac Lab also supports importing plane, mesh, and USD terrains and is NVIDIA's current robot-learning framework for Isaac Sim. [12]

CARLA should be the second future adapter for driving-oriented scenarios because it provides ray-cast LiDAR, semantic LiDAR, segmentation, instance IDs, vehicle maps, and AV-centric semantics. Semantic LiDAR already includes hit coordinates, cosine of incidence, instance ID, and semantic tag, which maps cleanly onto FailureCloud's label model. [13]

Gazebo should be the third future adapter for ROS-native simulation. Gazebo's sensor stack includes lidar, IMU, contact, depth, and GPU lidar support through the `gz-sensors` library, making it suitable for robotics teams that already standardize on SDF and ROS bridges. [14]

External integrations

Redis. Use Redis as scenario memory: store `scenario.json` plus embeddings and metadata in JSON documents; create a vector index and support KNN plus metadata filters so users can request "generate a harder version of the warehouse cup task." Redis officially supports vector indexes over JSON or hashes, KNN/range queries, and metadata filtering. [15]

Arize Phoenix. Instrument the pipeline with spans for prompt compilation, validation, preview, sim execution, labeling, export, and evaluation. Phoenix accepts traces over OTLP and provides LLM tracing plus evaluations, which is ideal for catching malformed scenario generations and export regressions. [16]

Nebius / Ultimate Bots. Treat this as a future orchestration target, not the primary runtime. Nebius now describes Physical AI Workbench as a YAML-manifest-driven orchestration layer with automated dependency management, retries, and artifact passing, built around Cosmos 3 and Isaac technologies. The adapter should therefore generate a **task manifest** rather than attempting custom execution logic. Public Ultimate Bots API docs were not readily discoverable in this research, so keep the export generic. [17]

Cosmos. Treat Cosmos as a high-fidelity world-generation backend that can consume controlled 3D scenes and annotations from Omniverse-based workflows, then increase photorealism and generate diverse rare conditions. Cosmos is not the MVP executor; it is a future backend for visual/world augmentation once deterministic scenario execution is already solved. [18]

Quality, testing, deployment, and performance

Testing plan

Recommended test layers:

Layer	What to test
Schema tests	Valid/invalid `scenario.json`, label JSON, export manifests
Prompt compiler tests	Golden prompts → deterministic normalized scenarios
Simulation smoke tests	Minimal scene runs for 50 steps in headless mode
Sensor tests	Output shapes, timestamps, and ranges
Label tests	Every exported object ID exists in registry
Reward tests	Known trajectories produce expected success/failure
Export tests	OpenPCDet folder passes validator; ROS export passes schema checks
UI tests	Preview renders valid scenes and shows result states

For CI, run headless PyBullet smoke tests on every PR. On Linux, Bullet's EGL renderer example is the right pattern for hardware-accelerated headless rendering; if EGL is unavailable, fall back to TinyRenderer. [19]

Deployment and CI recommendations

Recommended deployment shape:

- **Frontend:** Next.js or React app on Vercel/Netlify or container
- **Backend API:** FastAPI
- **Worker:** Python process or queue worker for long-running scenario execution
- **Storage:** Local filesystem for MVP; object storage later
- **Redis:** scenario memory + job status
- **Phoenix:** OTLP endpoint for traces and evals

Recommended CI jobs:

1. lint + typecheck
2. schema fixture validation
3. unit tests
4. headless PyBullet smoke sim
5. exporter validation
6. build Docker images
7. deploy preview

Performance targets

These are **engineering targets**, not platform guarantees:

Target	Recommended MVP bar
Prompt → valid scenario	< 5 s median
Browser preview render	< 1 s after validated scenario
PyBullet run initialization	< 3 s
10-second episode, no RGB	< 15 s wall-clock
10-second episode with RGB/depth/LiDAR at 5 Hz	< 40 s wall-clock
Export bundle generation	< 5 s after simulation completes
Sim smoke-test runtime in CI	< 90 s

The first optimization knob should be sensor cadence, not physics cadence. Keep physics at 240 Hz for stability if needed, but export sensors at 2–10 Hz for the demo. Use prompt caching for repeated compiler prefixes and Redis for scenario-reuse retrieval. [20]

Roadmap and hackathon demo checklist

Prioritized roadmap

Week	Milestone	Deliverables
1	Repo and schemas	monorepo, JSON Schemas, sample fixtures, CI skeleton
2	Claude compiler	prompt endpoint, strict schema outputs, validator
3	Preview	Three.js scene preview with placeholders
4	PyBullet scene loader	floor, robot, obstacles, assets, stepping loop
5	Sensor recorder	RGB, depth, segmentation, LiDAR-like raycast
6	Labeler and evaluator	3D boxes, collisions, reward, fake water model
7	Exporters	PyBullet bundle, OpenPCDet, MMDetection3D basic, ROS-style folder
8	Redis + Phoenix	scenario memory, tracing, eval dashboards
9	Future manifests	Isaac config stub, Nebius manifest, Cosmos manifest
10	Demo polish	end-to-end demo video, README, benchmarks, example scenarios

Final hackathon deliverables checklist

- [ ] Public repo with setup instructions

- [ ] `scenario.json` schema and sample scenarios
- [ ] Working prompt → scenario compiler
- [ ] Browser preview
- [ ] PyBullet simulation runner
- [ ] Sensor recorder producing RGB, depth, LiDAR-like arrays
- [ ] Auto-labeling and evaluation report
- [ ] Fake water success metric and reward graph
- [ ] OpenPCDet export folder
- [ ] ROS-style export folder with `PointCloud2` / `Image` / `CameraInfo` equivalents
- [ ] Redis-backed scenario memory demo
- [ ] Arize Phoenix tracing screenshot or live dashboard
- [ ] Nebius task manifest export
- [ ] Cosmos future manifest or architecture slide
- [ ] 3-minute demo showing prompt, preview, run, outputs, and export

The simplest demo scenario remains the strongest: **"Generate a warehouse robot unit test: carry a cup of water across a slippery floor while avoiding a dropped box."** If that one path works end to end, FailureCloud will already satisfy the core value proposition: natural language in, executable robot test case out.

---

1 3 PyBulletQuickstartGuide.md.html
https://github.com/bulletphysics/bullet3/blob/master/docs/pybullet_quickstart_guide/PyBulletQuickstartGuide.md.html?utm_source=chatgpt.com

2 Replicator — Omniverse Extensions
https://docs.omniverse.nvidia.com/extensions/latest/ext_replicator.html

4 9 Structured outputs - Claude API Docs
https://platform.claude.com/docs/en/build-with-claude/structured-outputs?utm_source=chatgpt.com

5 raw.githubusercontent.com
https://raw.githubusercontent.com/bulletphysics/bullet3/master/examples/pybullet/examples/batchRayCast.py

6 16 What is Arize Phoenix? - Phoenix
https://arize.com/docs/phoenix

7 Define tools - Claude API Docs
https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools?utm_source=chatgpt.com

8 OpenPCDet/docs/CUSTOM_DATASET_TUTORIAL.md at master · open-mmlab/OpenPCDet · GitHub
https://github.com/open-mmlab/OpenPCDet/blob/master/docs/CUSTOM_DATASET_TUTORIAL.md

10 raw.githubusercontent.com
https://raw.githubusercontent.com/bulletphysics/bullet3/master/examples/pybullet/examples/getCameraImageTest.py

11 raw.githubusercontent.com
https://raw.githubusercontent.com/ros2/common_interfaces/rolling/sensor_msgs/msg/PointCloud2.msg

12 RTX Lidar Sensor — Isaac Sim Documentation
https://docs.isaacsim.omniverse.nvidia.com/latest/sensors/isaacsim_sensors_rtx_lidar.html

13 Sensors reference - CARLA Simulator
https://carla.readthedocs.io/en/latest/ref_sensors/

14 Sensors — Gazebo jetty documentation
https://gazebosim.org/docs/latest/sensors/

15 Vector search concepts | Docs
https://redis.io/docs/latest/develop/ai/search-and-query/vectors/

17 Physical AI and Robotics on Nebius
https://nebius.com/solutions/physical-ai-and-robotics?utm_source=chatgpt.com

18 Scale Synthetic Data and Physical AI Reasoning with NVIDIA Cosmos World Foundation Models | NVIDIA Technical Blog
https://developer.nvidia.com/blog/scale-synthetic-data-and-physical-ai-reasoning-with-nvidia-cosmos-world-foundation-models/

19 raw.githubusercontent.com
https://raw.githubusercontent.com/bulletphysics/bullet3/master/examples/pybullet/examples/testrender_egl.py

20 Prompt caching - Claude API Docs
https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching